

# **UNIT- V**

## **Recurrent Neural Networks (RNN)**

# Unit-V - Recurrent Neural Networks

- Introduction, Recurrent Neurons, RNN architecture, Bidirectional RNN,
- LSTM: Properties, LSTM network architecture, LSTM Units, GRU, Encoder-decoder, sequence to sequence architecture,
- applications of RNN.

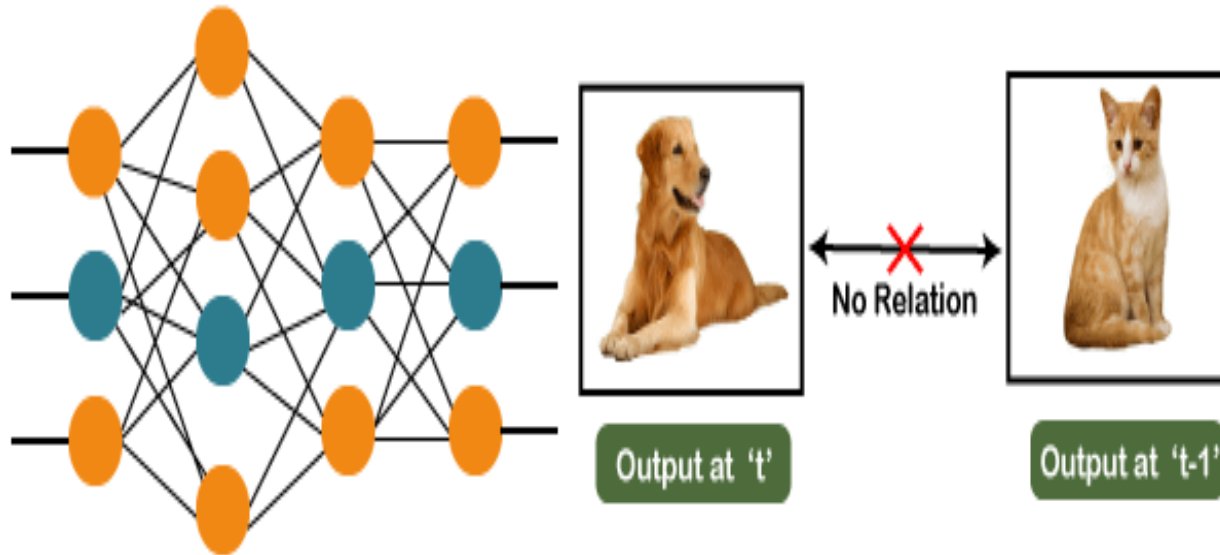
# Several neural networks can help solve different business problems.

- **Feed-Forward Neural Network:** Used for general Regression and Classification problems.
- **Convolutional Neural Network:** Used for object detection and image classification.
- **Deep Belief Network:** Used in healthcare sectors for cancer detection.
- **RNN:** Used for speech recognition, voice recognition, time series prediction, and natural language processing.

## **Problem with Feedforward NN and CNN**

- Model reads each of these inputs independently.
- There is no dependence between the inputs and the size of the input is fixed.
- There is no temporal or sequential relationship between inputs.

# Feed forward Neural Networks



The output at the time ' $t$ ' is **independent** of output at a time ' $t-1$ '.

**No relation** between the new output and the previous output.

# Challenges of ANNs for NLP Tasks

## 1. Fixed Input/Output Size

- NLP tasks often involve **variable-length text** (e.g., sentences, paragraphs).

## 2. High Dimensionality of Text Data

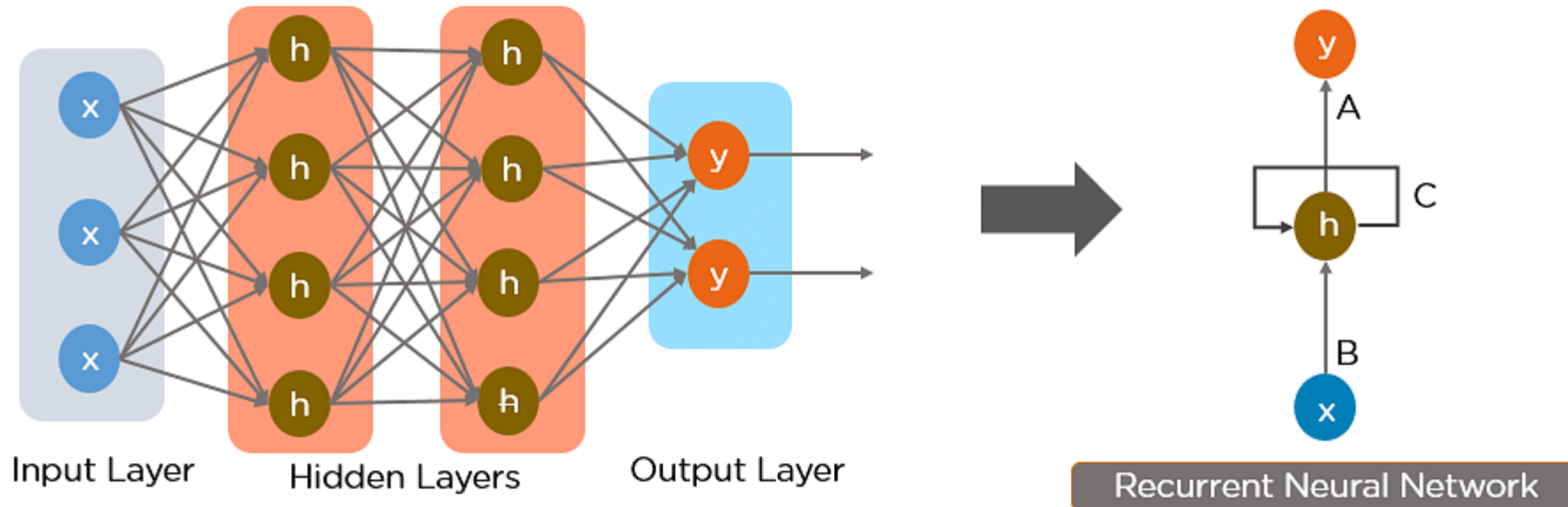
- Text data, when converted into numerical form (e.g., one-hot encoding), results in sparse and high-dimensional vectors.
- This increases **computational cost** and makes it difficult for simple ANNs to capture **meaningful relationships**.

## 3. Lack of Long-Term Dependencies

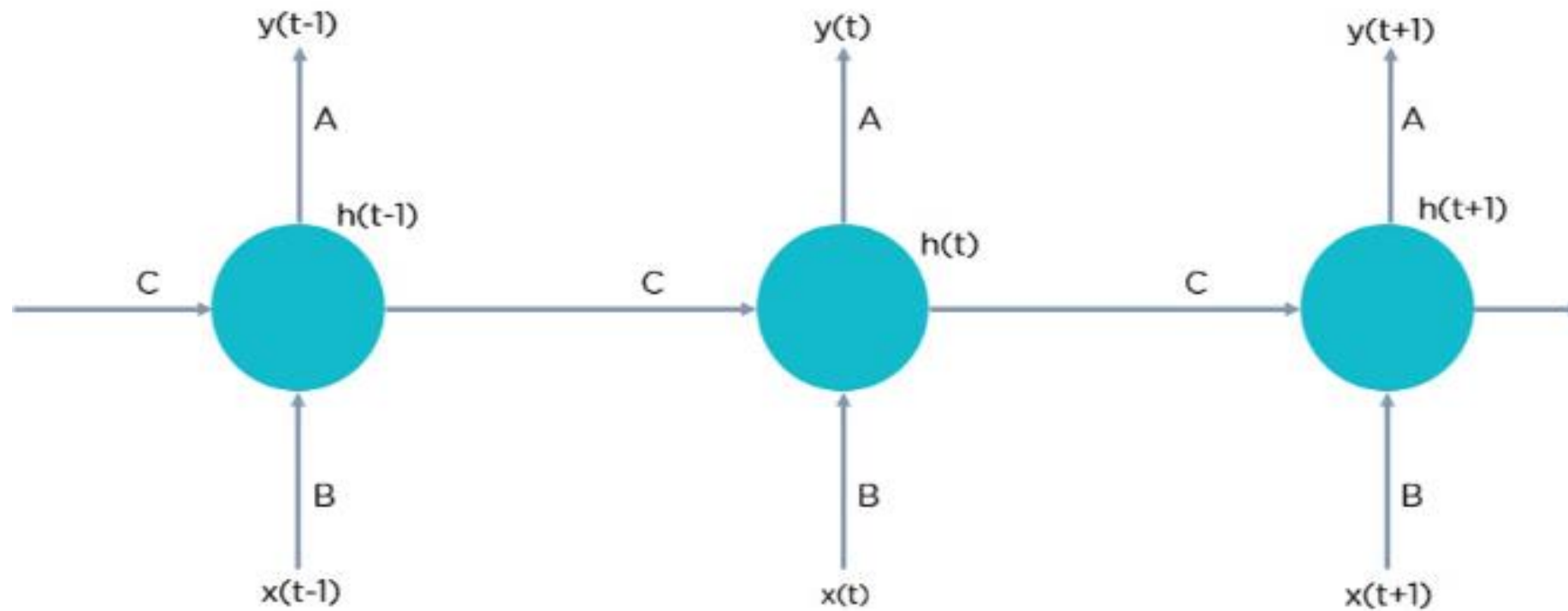
- NLP tasks often require understanding **long-range dependencies** (e.g., in long sentences or documents).
- Simple **ANNs do not have memory mechanisms** like **Recurrent Neural Networks (RNNs)** or **Transformers** to capture context over long distances.

# What Are Recurrent Neural Networks (RNN)?

- **Recurrent Neural Networks (RNNs)** are a type of artificial neural network designed to process **sequences of data**.
- RNN works on the principle of saving the output of a particular layer and feeding this back to the input in order to predict the output of the layer.



# Fully connected Recurrent Neural Network

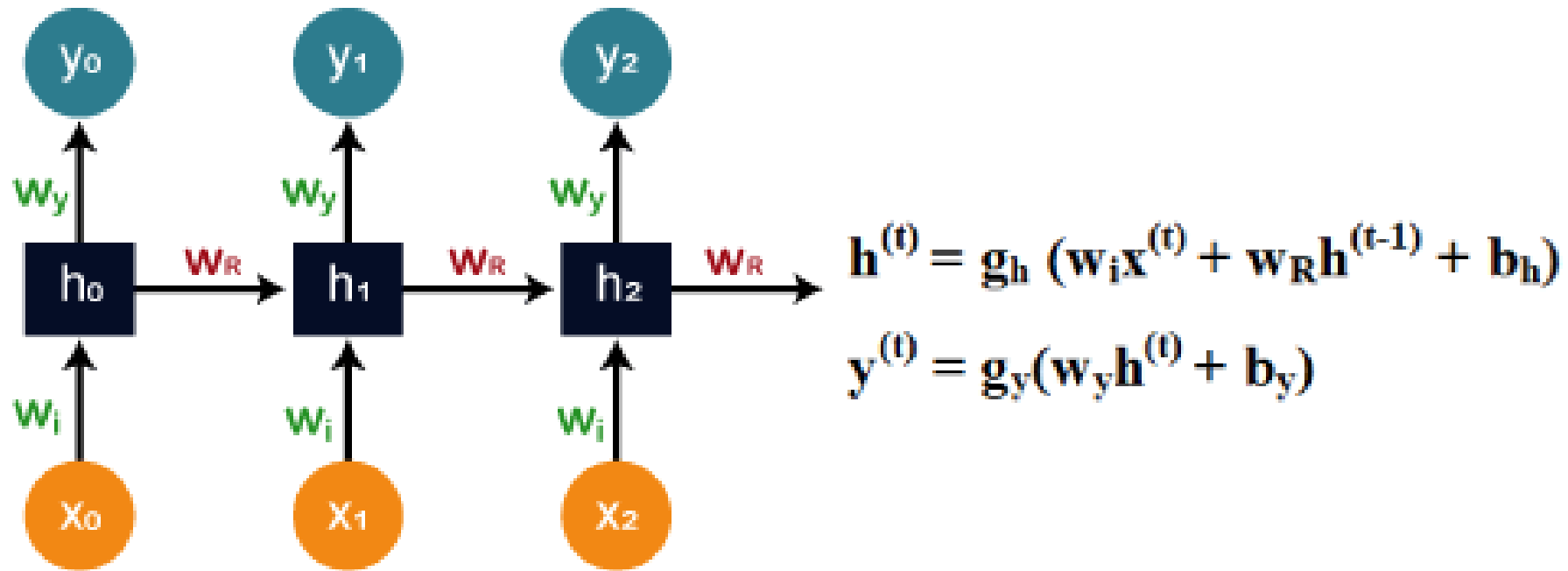


$$h(t) = f_c(h(t-1), x(t))$$

$h(t)$  = new state  
 $f_c$  = function with parameter  $c$   
 $h(t-1)$  = old state  
 $x(t)$  = input vector at time step  $t$

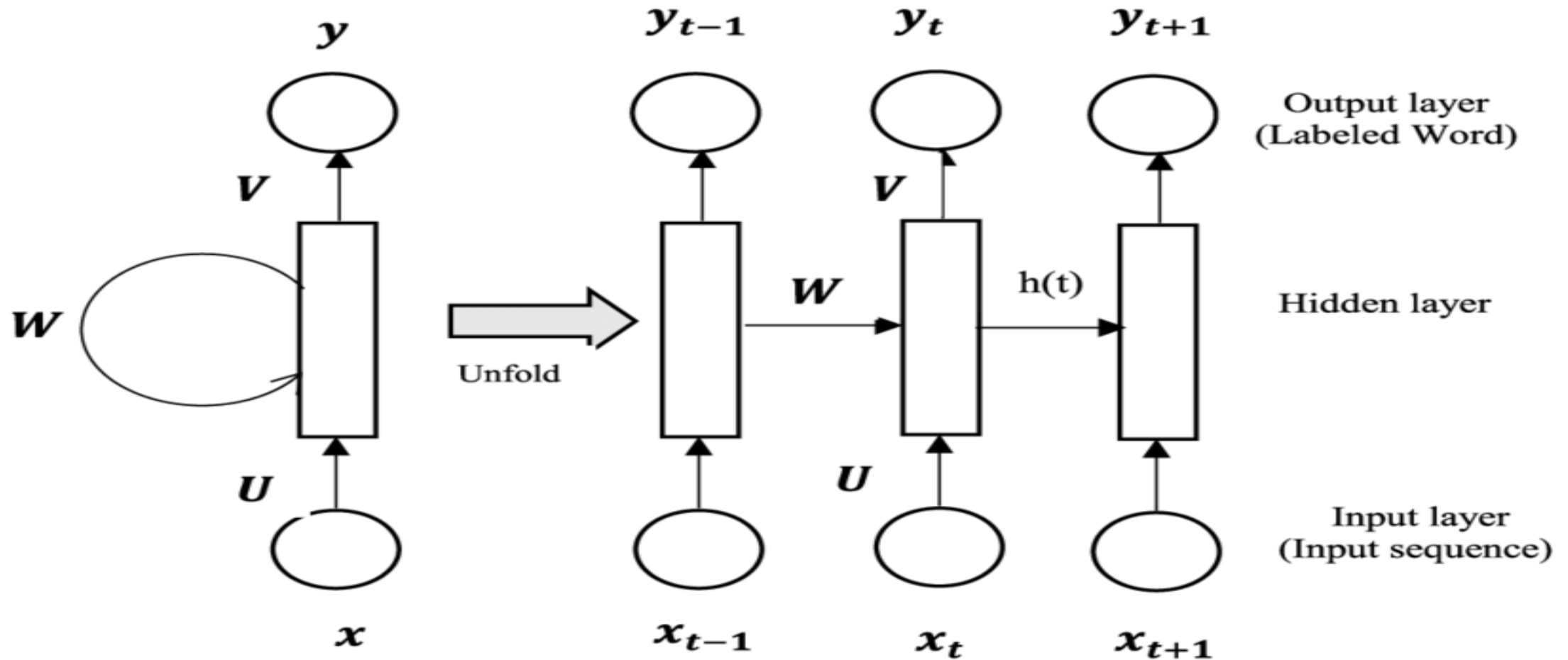


# Hidden layer & Output layer computation

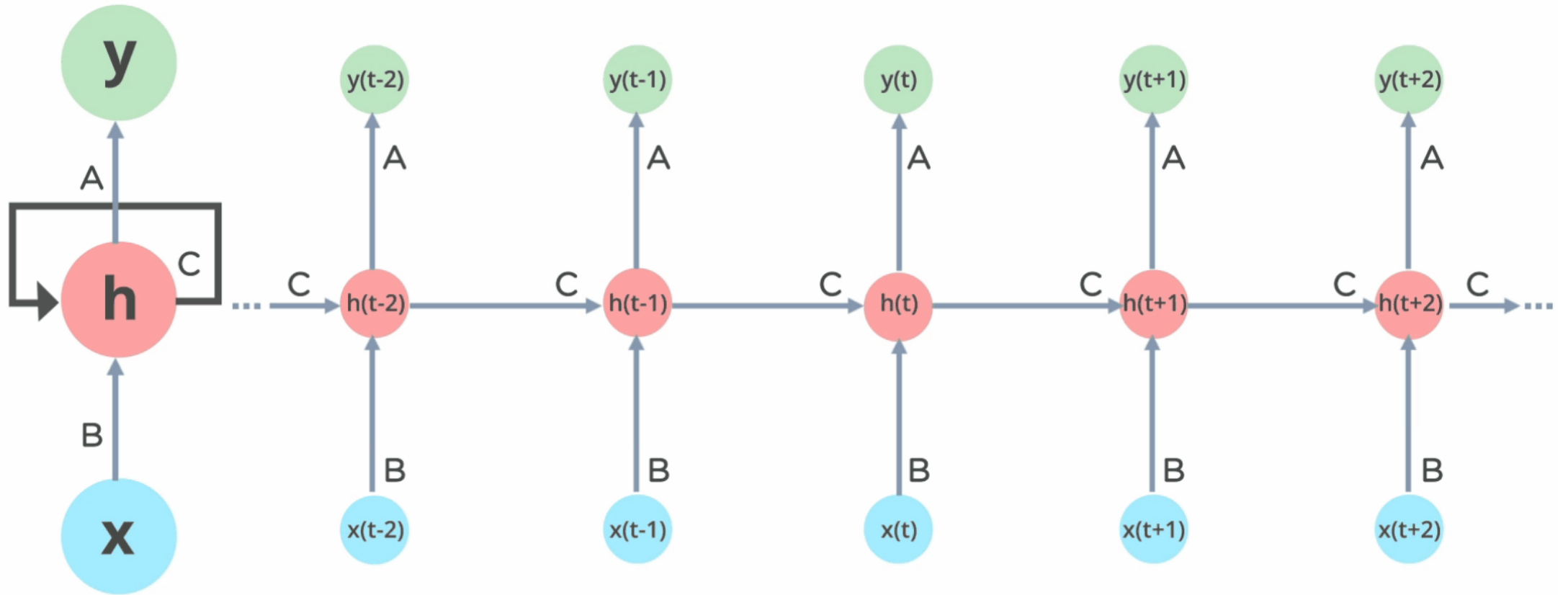


$g_h$  is an activation function (e.g., tanh or ReLU)  
 $g_y$  is the activation function for the output (e.g., softmax for classification),

# Folded and unfolded regular recurrent neural networks with single hidden recurrent layer



# How does RNN works?



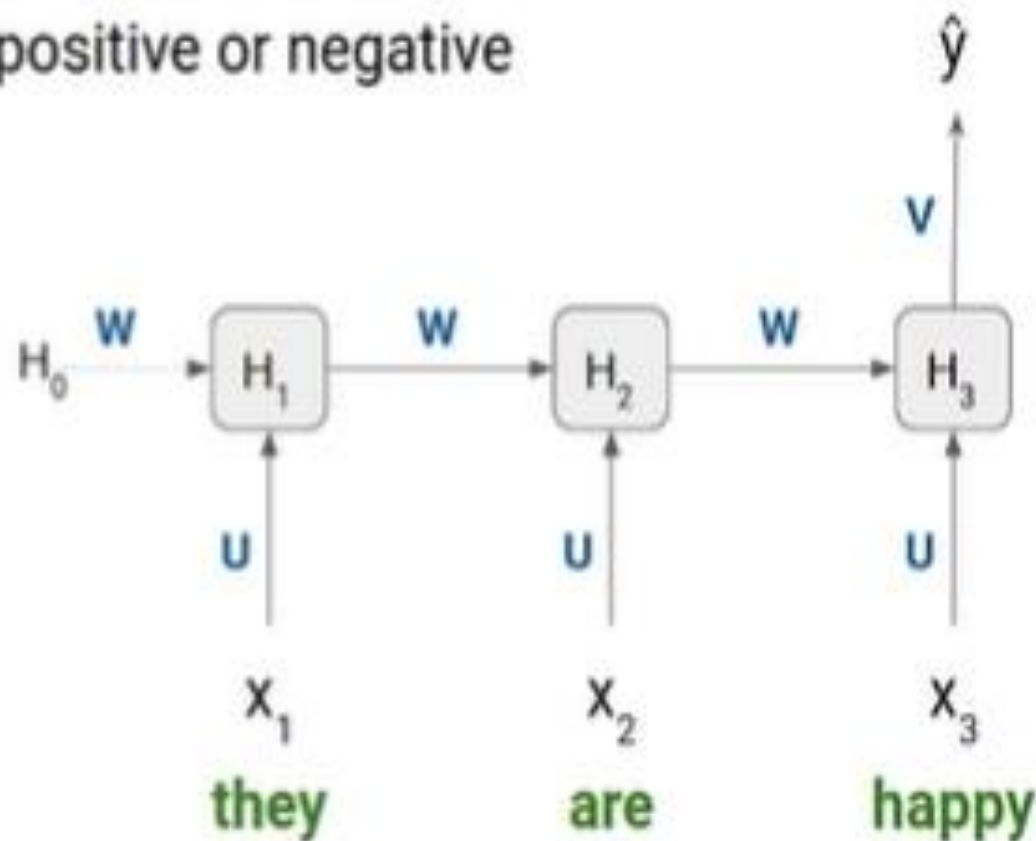
# Artificial Neuron Vs Recurrent Neuron

| Feature            | Artificial Neuron                         | Recurrent Neuron                   |
|--------------------|-------------------------------------------|------------------------------------|
| Network Type       | Feedforward Neural Network (FNN)          | Recurrent Neural Network (RNN)     |
| Memory             | No memory, processes inputs independently | Has memory, depends on past inputs |
| Information Flow   | One direction (input → output)            | Loops back (feedback mechanism)    |
| Usage              | Image processing, classification          | Sequential data, time-series, NLP  |
| Mathematical Model | $y = f(Wx + b)$                           | $h_t = f(Wx_t + Uh_{t-1} + b)$     |

# Functioning of RNN - Forward Propagation

**Task:** Classify sentiment of the text as positive or negative

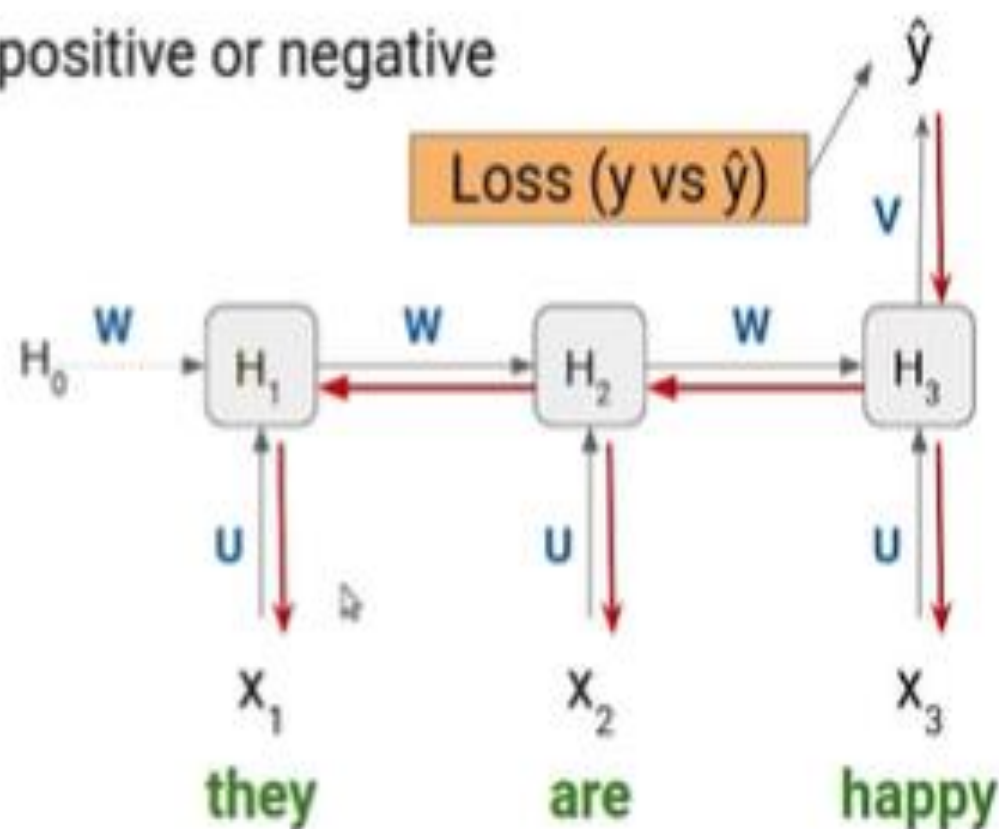
- Input sentence: "they are happy"
- First token, 'they', will be passed as input at timestep 1
- Second token, 'are', will be passed at timestep 2 and so on
- Output obtained at last timestep.



# Functioning of RNN - Back Propagation

**Task:** Classify sentiment of the text as positive or negative

- Loss ( $L$ ) is calculated at the final timestep
- $\partial L / \partial V$ ,  $\partial L / \partial W$  and  $\partial L / \partial U$  are computed
- Weight matrices  $W$ ,  $U$ , and  $V$  are updated



# Why Recurrent Neural Networks?

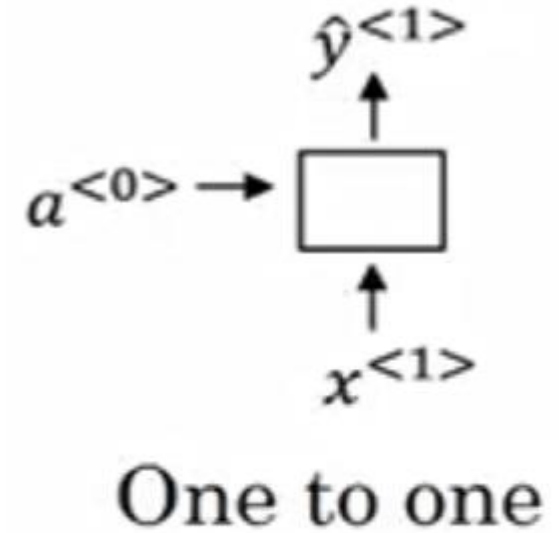
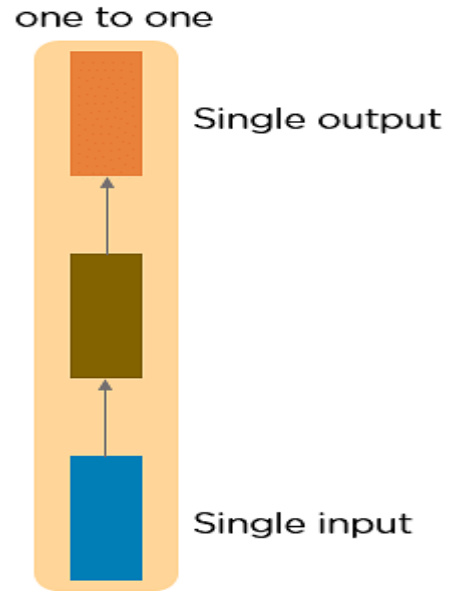
Issues in the feed forward neural network:

- Cannot handle sequential data
- Considers only the current input
- Cannot memorize previous inputs
- An RNN can handle sequential data, accepting the current input data, and previously received inputs.

**RNNs** can memorize previous inputs due to their **internal memory**.

# Types of RNNs

One-One RNN (Vanilla RNN)  
One-Many RNN  
Many-One RNN  
Many-Many RNN



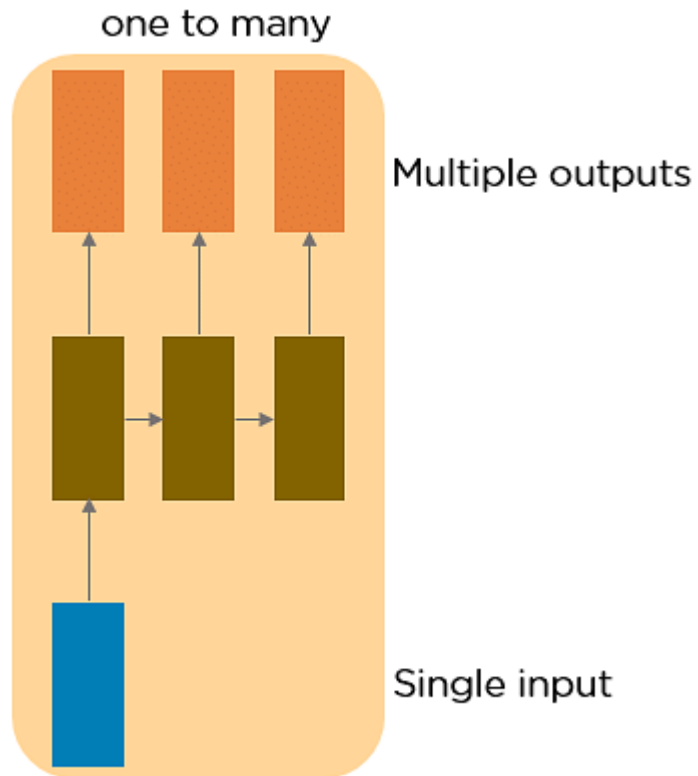
It is used to solve general machine learning problems that have only **one input and output**.

**Example:**

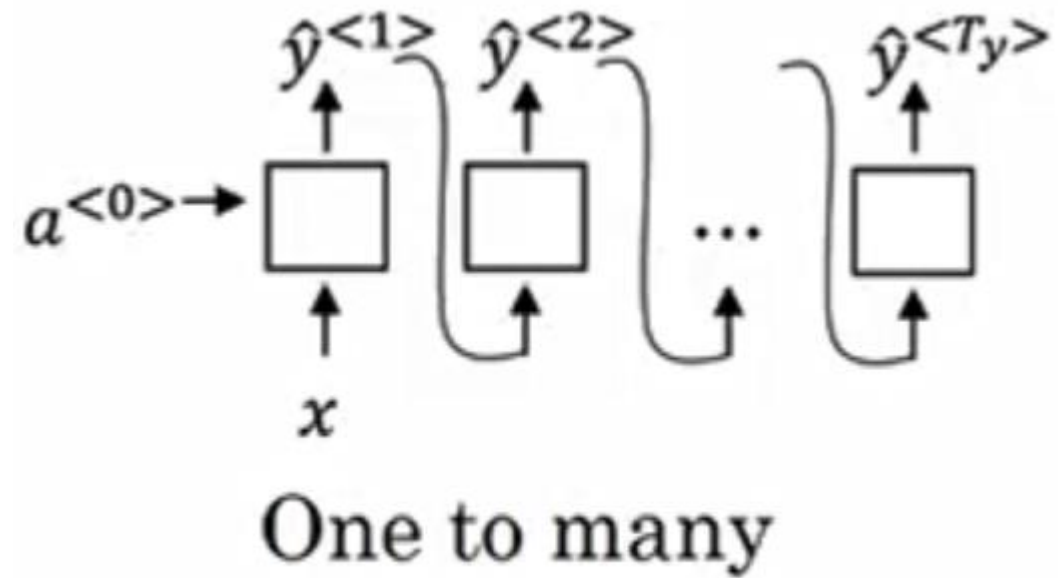
- **Input:** A single image
- **Output:** A label (e.g., "Cat")



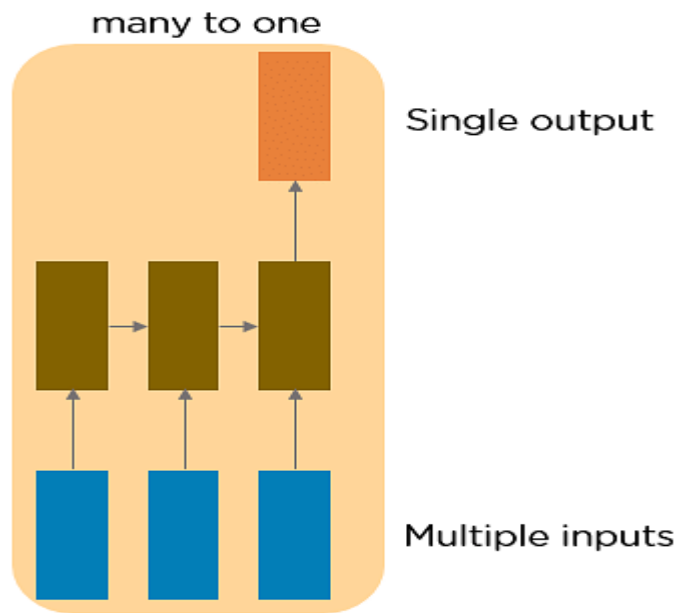
# One to many



*Example: The image is sent into Image Captioning, which generates a sentence of words.*



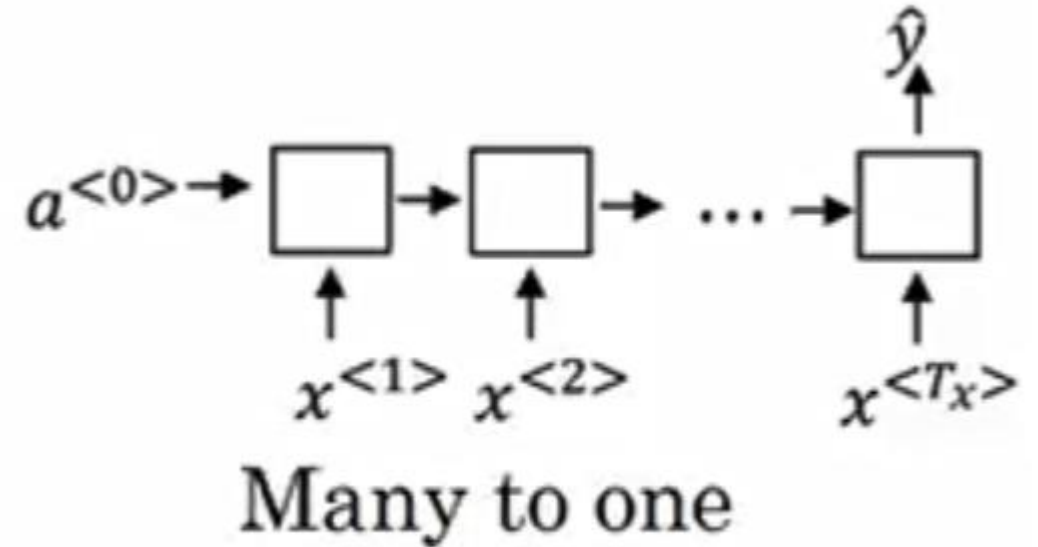
# Many-to-One RNN:



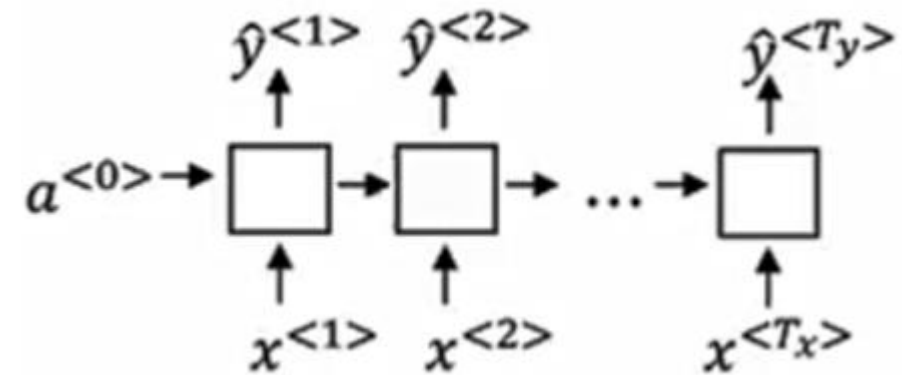
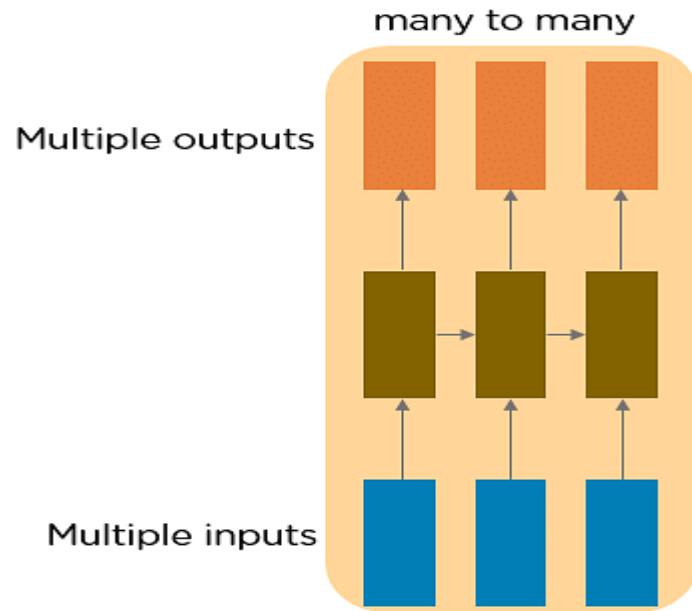
Example : Sentiment analysis  
(input: sentence, output: positive/negative sentiment)

Input: "The movie was amazing!"

Output: "Positive"



# Many-to-Many RNN(Synchronous):



**Example: Parts of speech tagging**

• **Input:** "I love AI"

• **Output:** "PRON VERB NOUN"

|            |    |    |    |    |
|------------|----|----|----|----|
| Time Step: | t1 | t2 | t3 | t4 |
| Input:     | X1 | X2 | X3 | X4 |
| Output:    | Y1 | Y2 | Y3 | Y4 |

(Same length as input)

**Output is generated at the same time as input processing.**

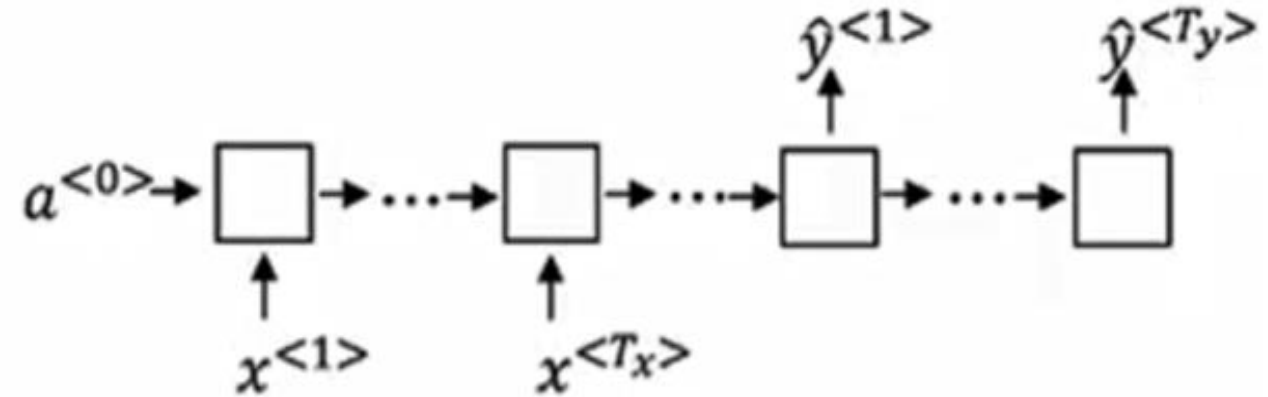
# Many-to-Many (Asynchronous)

- The input and output sequences have different lengths.
- The model first reads the full input sequence (Encoder) and then generates an output sequence (Decoder).

Machine Translation (e.g., English → French):

• Input: "How are you?"

• Output: "Comment ça va?"



Input: X1 X2 X3 X4 X5 (Encoder processes **all** inputs first)

↓

Output: Y1 Y2 Y3 Y4 (Decoder generates outputs **after** processing **input**)

**The output sequence begins only after the full input sequence is processed.**

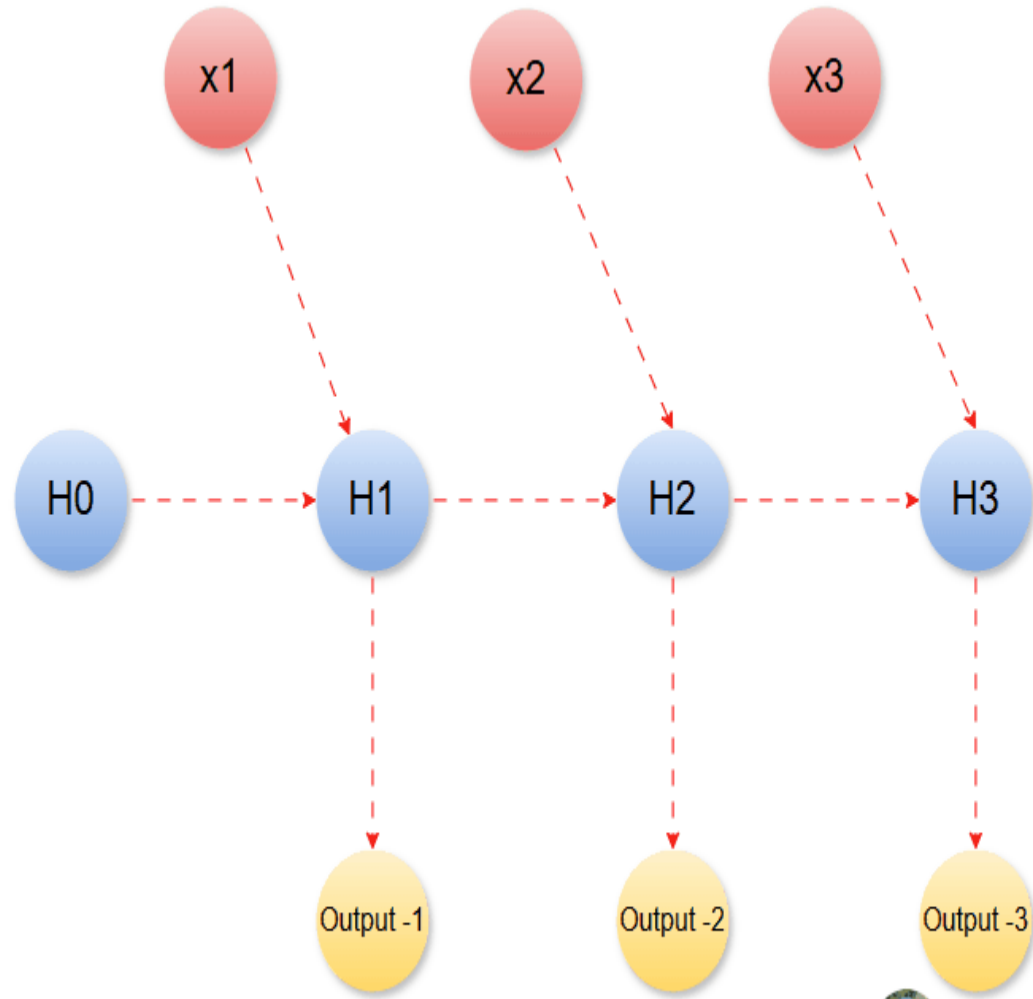
# Training in RNN

- The major steps involved in RNN training are
- data preparation(input)
- forward propagation
- loss computation
- backpropagation through time (BPTT) and weight updates.

# Data Preprocessing(Preparation)

- Before training, the sequential data must be preprocessed properly.
- **Key tasks:**
  - Tokenization (for NLP) or Normalization (for time-series data).
  - Padding sequences to ensure uniform length.
  - Train-test split (e.g., 80%-20%).
  - Batching to improve efficiency

# Forward Propagation



At each time step  $t$ , the computations are as follows:

## 1. Hidden State Update:

$$h_t = f(W_x x_t + W_h h_{t-1} + b_h)$$

- $h_t$ : Current hidden state.
- $W_x$ : Input-to-hidden weights.
- $W_h$ : Hidden-to-hidden weights.
- $h_{t-1}$ : Previous hidden state.
- $b_h$ : Bias term.
- $f$ : Activation function (e.g., tanh).

## 2. Output Calculation:

$$o_t = W_o h_t + b_o$$

- $W_o$ : Hidden-to-output weights.
- $b_o$ : Output bias.

## 3. Prediction (for classification tasks):

$$\hat{y} = \sigma(o_t) \quad (\text{for binary classification})$$

- $\sigma$ : Sigmoid activation function.

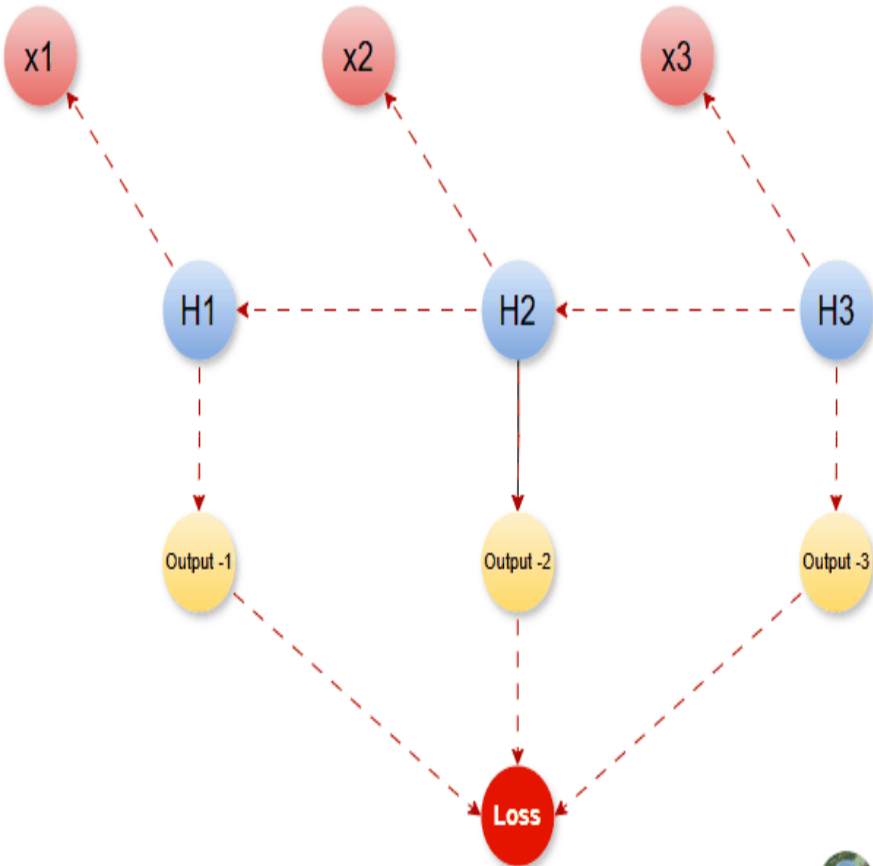
# Compute Loss

- The loss function measures how different the predicted output is from the actual target.
- Common loss functions:
  - **Cross-Entropy Loss** (for classification tasks)
  - **Mean Squared Error (MSE)** (for regression tasks)

$$\mathcal{L} = \sum_t \text{Loss}(y^{(t)}, \hat{y}^{(t)})$$



# Backpropagation Through Time (BPTT)



1. **Inputs:**  $x_1, x_2, x_3$  are connected to their corresponding hidden states  $H_1, H_2, H_3$ .
2. **Feedback Connections:** Hidden states are connected sequentially as  $H_0 \rightarrow H_1 \rightarrow H_2 \rightarrow H_3$ , ensuring the flow of context across time steps.
3. **Outputs:**  $o_1, o_2, o_3$  are derived from hidden states and contribute to the final loss calculation.
4. **Gradient Flow:** Backward arrows represent the flow of gradients during backpropagation, updating weights across all time steps.

$$W_{new} = W_{old} - \alpha \underbrace{\frac{dJ}{dW}}_{\text{gradient}}$$

# Handling Vanishing & Exploding Gradients

- **Vanishing Gradient:**

- Gradients become too small, preventing weight updates.
- Solution: Use **LSTM/GRU** instead of vanilla RNN, **ReLU activation**

- **Exploding Gradient:**

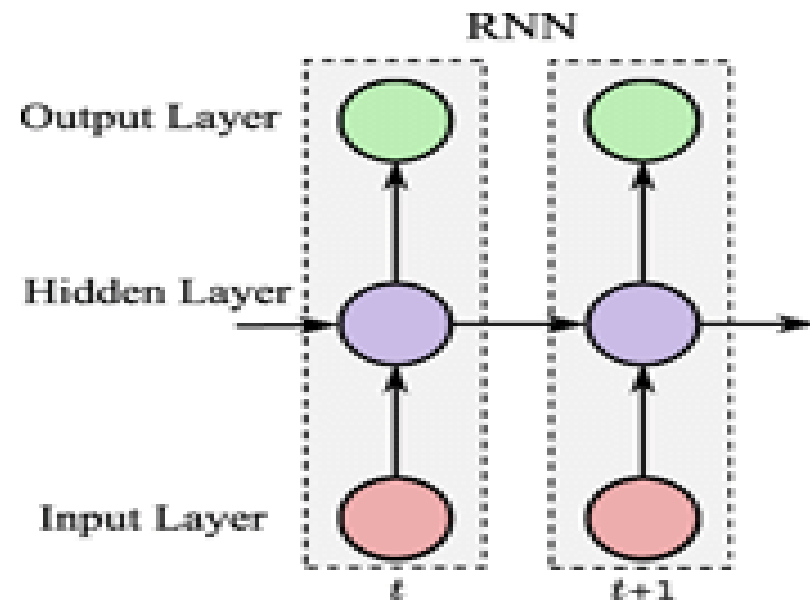
- Gradients grow too large, causing unstable updates.
- Solution: Use **gradient clipping** (e.g., limit gradient values).

# Bi-directional RNN (BRNN)

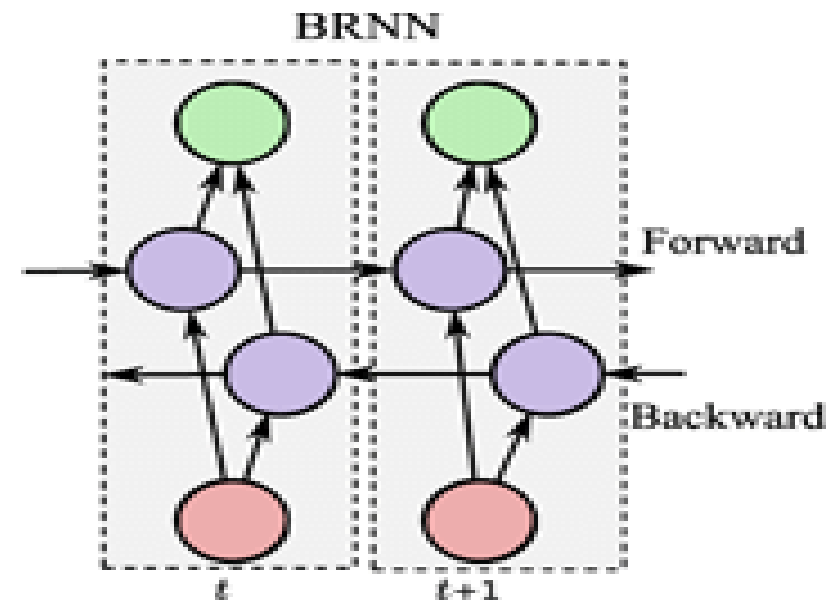
- Standard RNNs process data **only in one direction** (past → future).

**Example: I like apple. It is very healthy.**

- However, in many real-world problems, Here if we use normal RNN it may get confuse what we referring is it **fruit or company** by 1st line.
- **BRNN** will get its context easily as it works by moving in **both directions**.
- **future context** is also important to understand the full meaning of a sequence.
- This enables the network to utilize both past and future context when making predictions.



**RNN only has a forward hidden layer**



**BRNN consists of a forward hidden layer as well as a backward hidden layer**

# Working Mechanism of BRNN

A BRNN consists of **two RNNs**:

**1. Forward RNN** → Processes the sequence from **left to right** (past to future).

**2. Backward RNN** → Processes the sequence from **right to left** (future to past).

Each time step  $t$  has:

- An input  $x^{(t)}$
- A forward hidden state  $h_f^{(t)}$
- A backward hidden state  $h_b^{(t)}$
- A final hidden state  $h^{(t)}$  formed by combining both states
- An output  $y^{(t)}$

Forward Pass (Left to Right):

$$h_f^{(t)} = g_h(W_i x^{(t)} + W_f h_f^{(t-1)} + b_f)$$

Backward Pass (Right to Left):

$$h_b^{(t)} = g_h(W_i x^{(t)} + W_b h_b^{(t+1)} + b_b)$$

Final Hidden State (Combination of Both Directions):

$$h^{(t)} = \text{concat}(h_f^{(t)}, h_b^{(t)})$$

Output Calculation:

$$y^{(t)} = g_y(W_y h^{(t)} + b_y)$$

# Applications of BRNN

## 1.Sentiment Analysis

Social media analysis, product reviews, and customer feedback processing.

## 2. Named Entity Recognition

Example : Identifying "New York" as a place requires context from both sides.

## 3.Machine Translation

A word's meaning depends on the entire sentence structure.

## 4. Speech Recognition

Understanding a word based on the whole sentence, not just past words.

# Advantages of BRNNs

- **Enhanced context understanding:** Uses both past and future data for better predictions.
- **Better handling of variable-length sequences:** More flexible compared to standard RNNs

# Challenges of BRNN

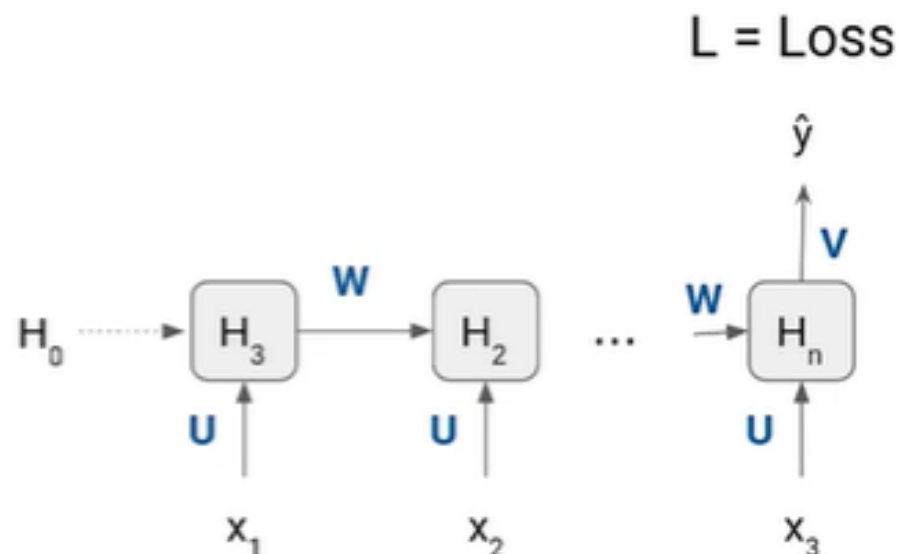
- **High computational cost:** Requires twice processing compared to unidirectional RNNs.
- **Longer training time:** More parameters to optimize making convergence slow.

# Vanishing Gradient

- $\partial L / \partial W \rightarrow \partial H_n / \partial W \rightarrow \partial H_{n-1} / \partial W \dots \partial H_0 / \partial W$
- $\partial L / \partial U \rightarrow \partial H_n / \partial U \rightarrow \partial H_{n-1} / \partial U \dots \partial H_0 / \partial U$
- If gradients  $< 1$

then  $\partial L / \partial W$  and  $\partial L / \partial U$  will be infinitesimally small

$$(0.5)^{10} = 0.00097$$





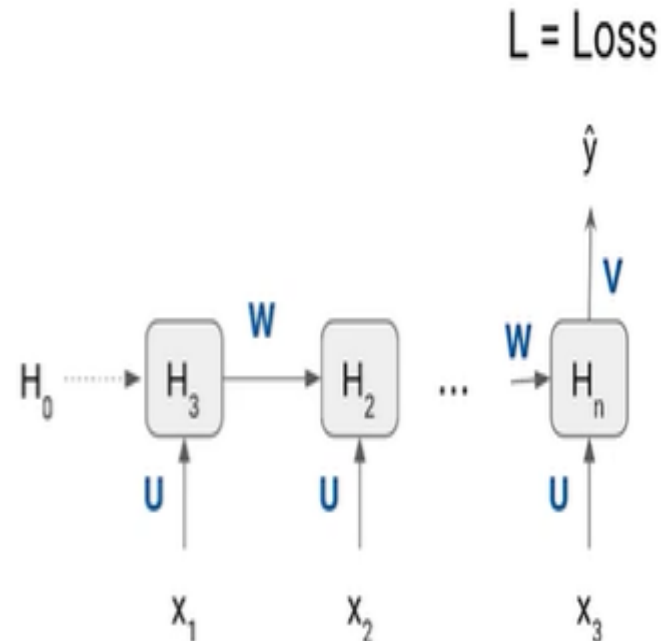
# Vanishing Gradient

- $\partial L / \partial W \approx \partial L / \partial U \approx 0$

- $$\begin{aligned} W &= W - \alpha (\partial L / \partial W) \\ U &= U - \alpha (\partial L / \partial U) \end{aligned}$$


updating weights

- $$\begin{aligned} W_{\text{before}} &\approx W_{\text{after}} \\ U_{\text{before}} &\approx U_{\text{after}} \end{aligned}$$

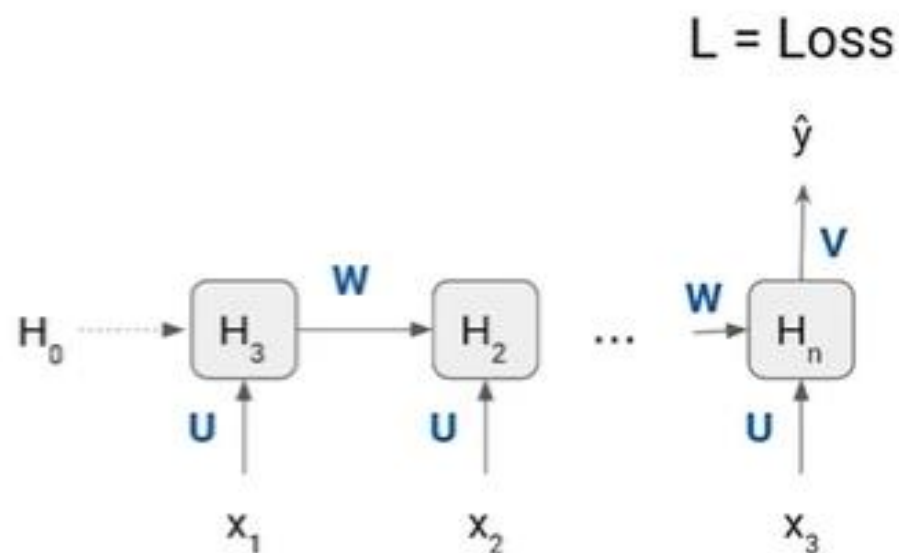


# Exploding Gradient

- $\partial L / \partial W \rightarrow \partial H_n / \partial W \rightarrow \partial H_{n-1} / \partial W \dots \partial H_0 / \partial W$
- $\partial L / \partial U \rightarrow \partial H_n / \partial U \rightarrow \partial H_{n-1} / \partial U \dots \partial H_0 / \partial U$
- If gradients  $> 1$

then  $\partial L / \partial W$  and  $\partial L / \partial U$  are very large

$$(1.5)^{10} = 57.665$$



# Issues with RNN

- **Vanishing Gradient**

RNN does not perform well for long sentences

Eg: The **writer** of the books **is**  
The writer of the **books** **are**

- **Exploding Gradient**

- Gradients are large
- Poor predictions

# Issues with RNN

How to mitigate exploding and vanishing gradients?

| Exploding Gradients | Vanishing Gradients |
|---------------------|---------------------|
| Gradient Clipping   | LSTM or GRU         |

# Long Short-Term Memory(LSTM) Networks

- **LSTM** networks are an **extension** of **RNNs**.
- Introduced to handle situations where RNNs fails (Vanishing Gradient Problem).
- RNNs are **incapable** of handling “**long-term dependencies**”.

# LSTM Units(The Building Blocks of LSTM Networks)

**Each LSTM unit consists of:**

**Forget Gate** → Decides what information to discard from memory.

**Input Gate** → Decides what new information to store.

**Cell State Update** → Maintains long-term memory.

**Output Gate** → Decides what information to pass forward.

Think of an LSTM unit like a diary:

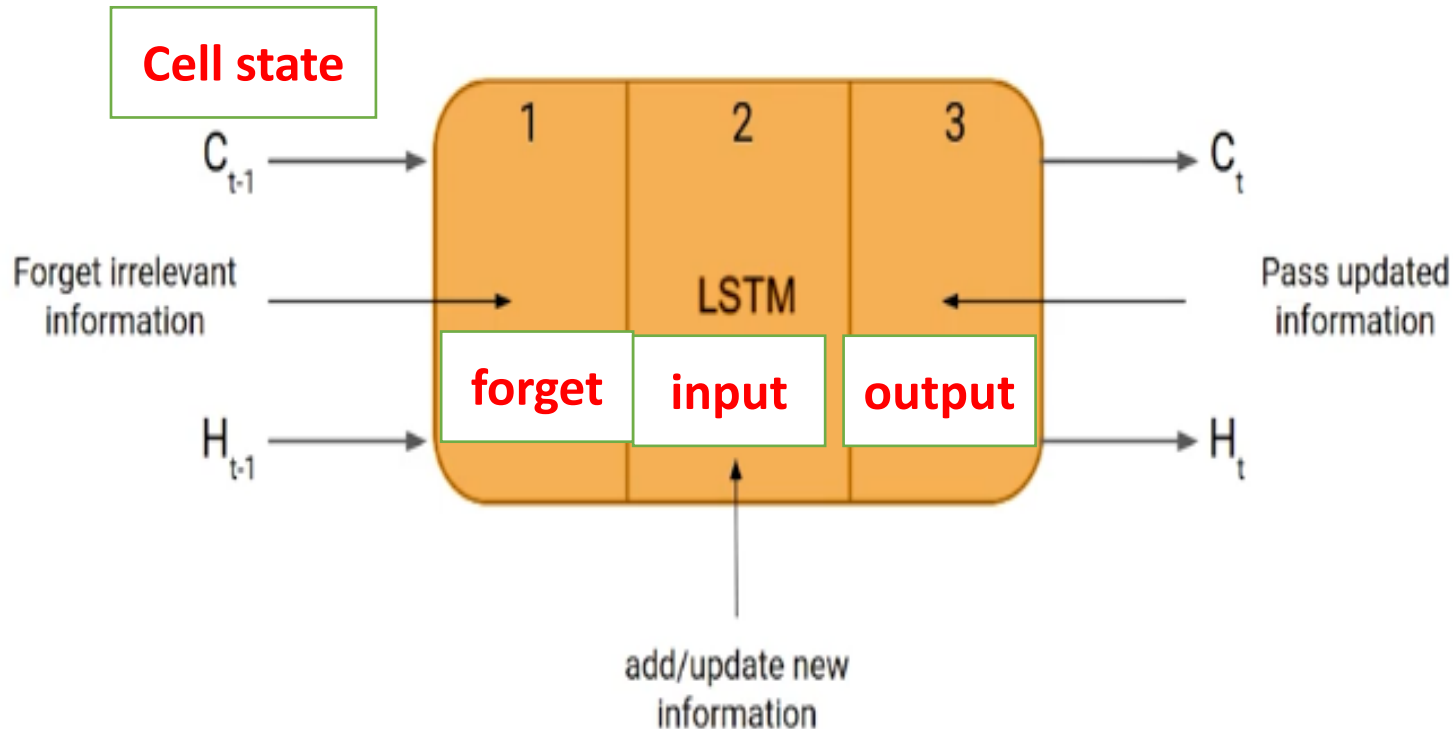
**Forget Gate** = Erases old, unimportant notes.

**Input Gate** = Writes new important notes.

**Cell State** = The diary itself, storing important details.

**Output Gate** = Reads out relevant notes at the right time.

# LSTM



LSTM mainly has **two states**:

1. **Cell State ( $C_t$ )** → Long-term memory

2. **Hidden State ( $h_t$ )** → Short-term memory / Output

## Cell State ( $C_t$ ) – Long-Term Memory

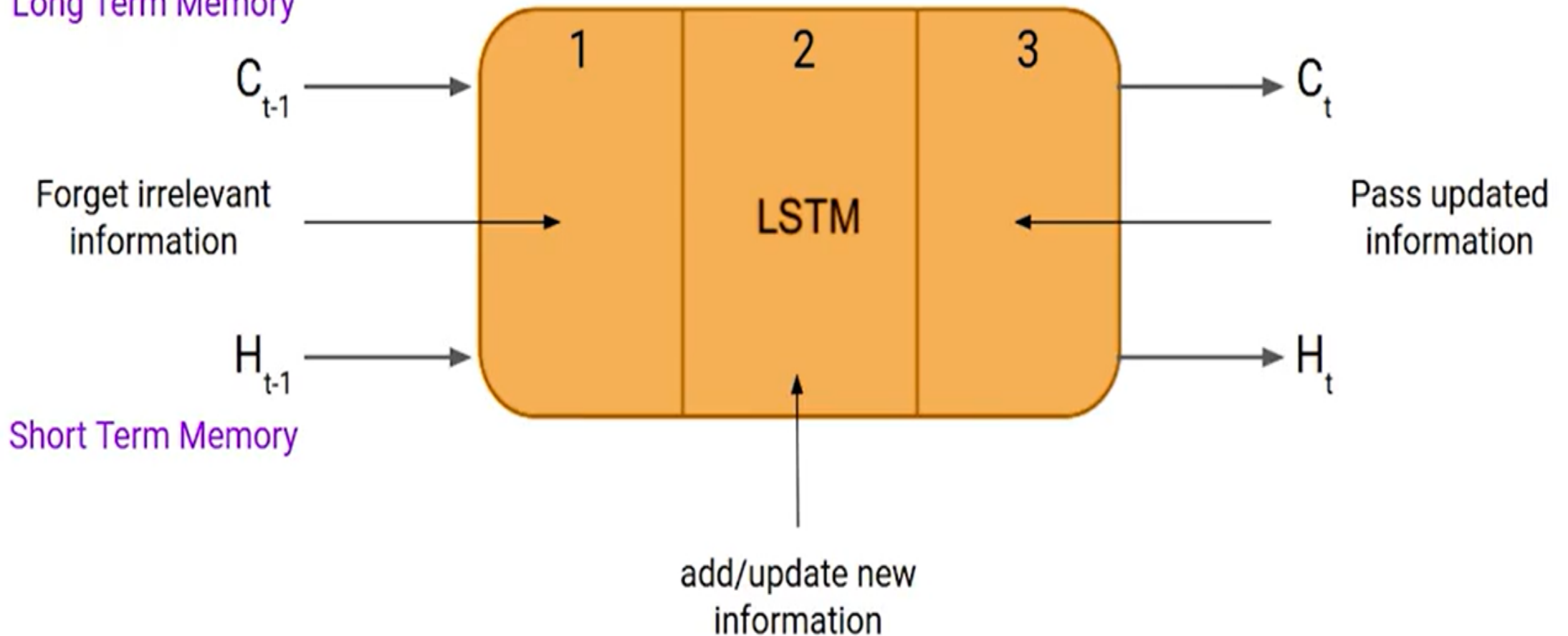
Think of this as a **conveyor belt** running through time.

- It carries important information from the past.
- It can remember information for a long time.
- It changes slowly and carefully.

## Hidden State ( $h_t$ ) – Short-Term Memory

- This is the output at each time step.
- It contains current information.
- It is passed to the next time step.
- It is also used to make predictions.

Long Term Memory



**Forget Gate** → What to remove from memory

**Input Gate** → What new information to add

**Output Gate** → What to send as output



# Forget Gate

LSTM



LSTM

Bob is a nice person. Dan on the other hand is evil.

# Input Gate



LSTM

Bob knows **swimming**. He told me over the **phone**  
that he had served the **navy** for 4 long years.

Input Gate

# Output Gate

The Output Gate decides:

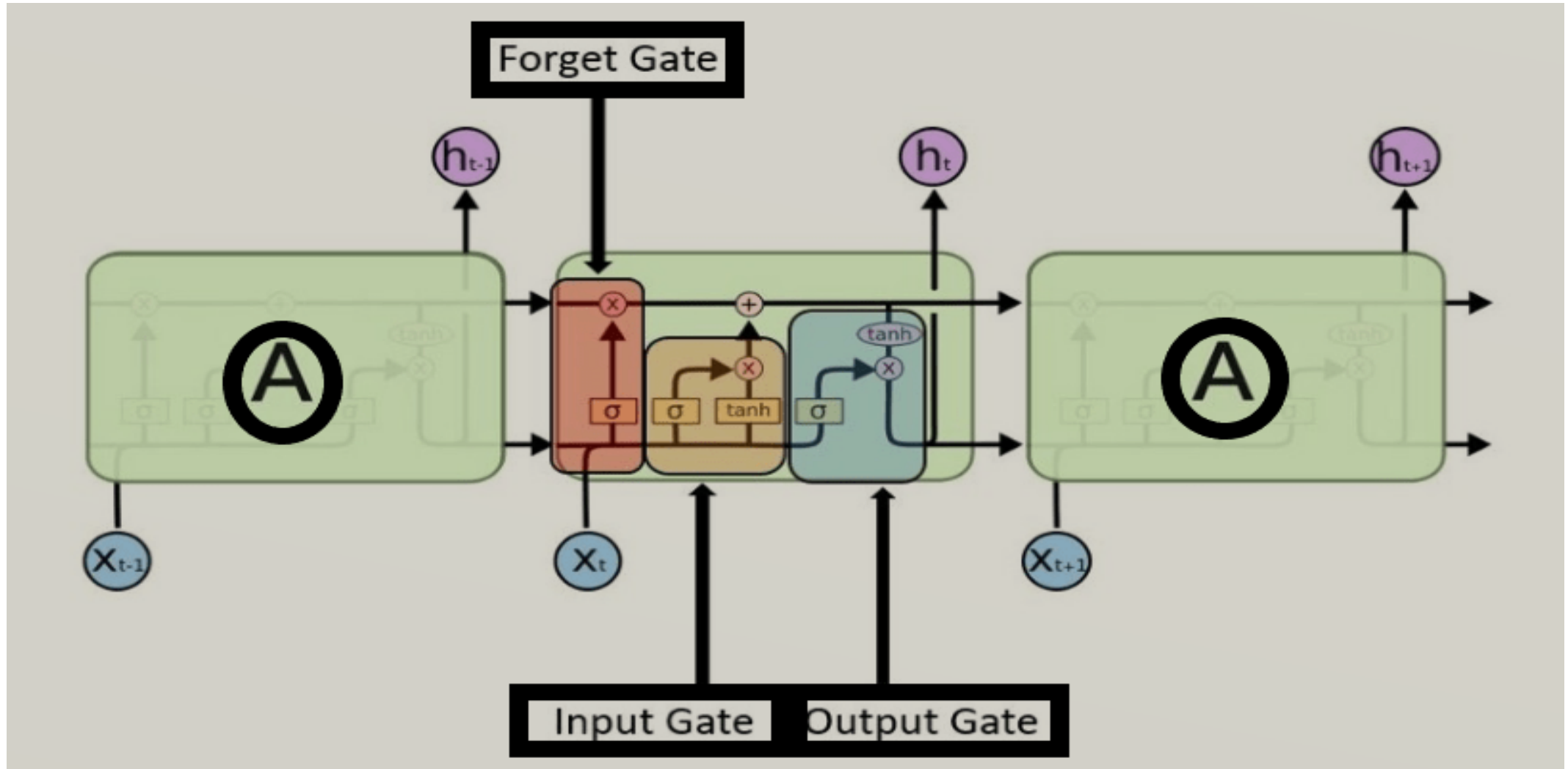
How much information from the cell state should be sent as the output (hidden state).



Bob single handedly fought the enemy and died for his country. For his contributions, brave \_\_\_\_\_.

Output Gate

# LSTM Architecture



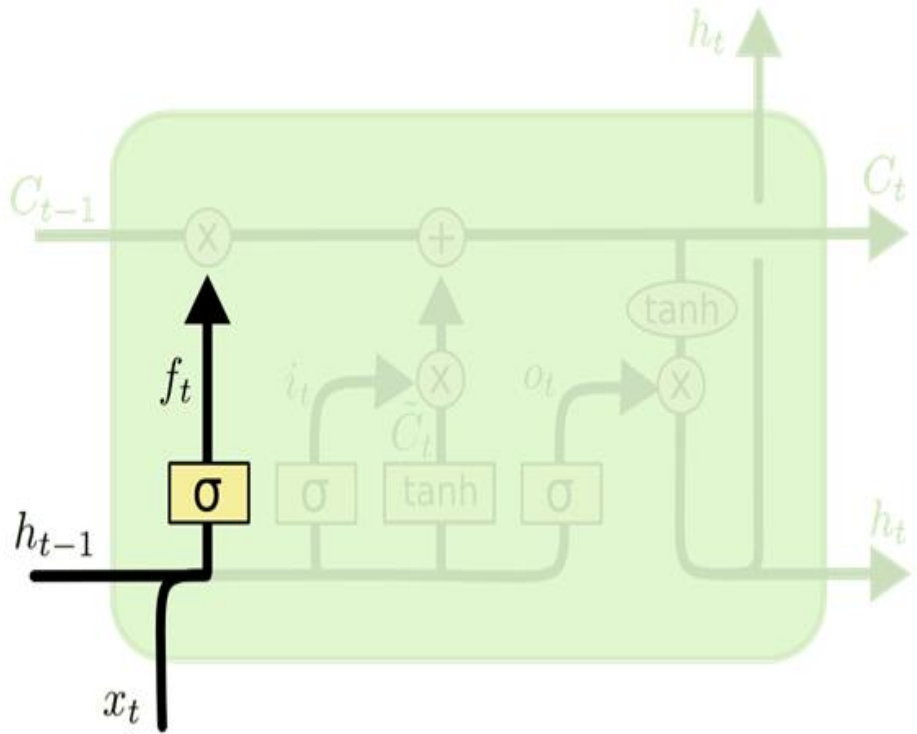
# Step 1: Inputs to the LSTM Cell

- At each time step  $t$ , the LSTM receives two inputs:

1. Current Input ( $X_t$ ) → The word or data at the current step.
2. Previous Hidden State ( $h_{t-1}$ ) → Memory from the previous step.

Instead of just passing a hidden state like a normal RNN, LSTM maintains a **cell state** to remember long-term dependencies.

## Step 2: Forget Gate



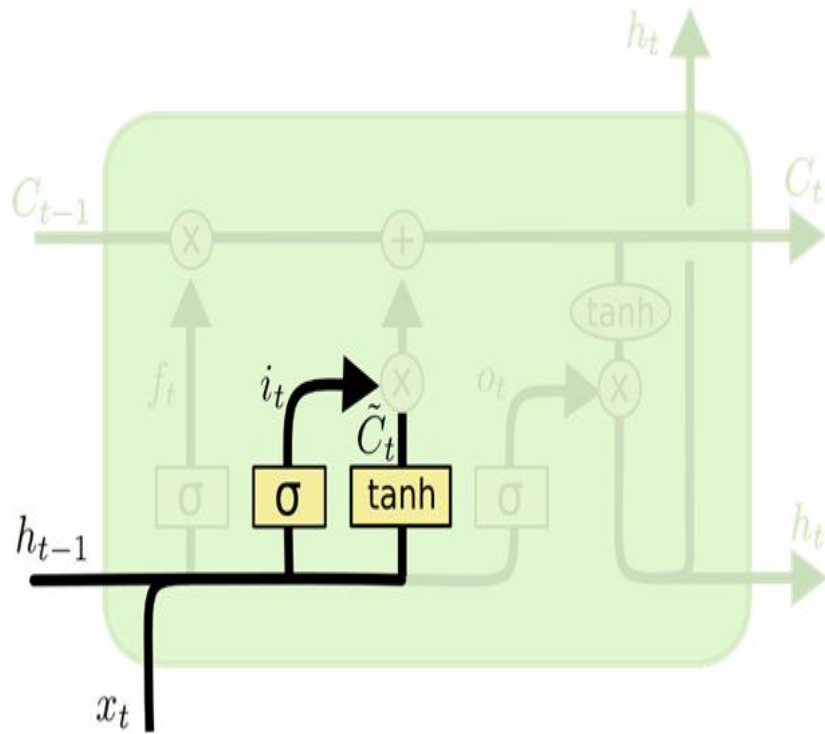
This gate decides what old information should be discarded from the cell state.

- Uses a sigmoid ( $\sigma$ ) function (values between 0 and 1) to decide:
  - 0  $\rightarrow$  Forget the information
  - 1  $\rightarrow$  Keep the information
- Formula:

$$f_t = \sigma(W_f \cdot [h_{t-1}, X_t] + b_f)$$

**"I love this movie, but it was too long."**

## Step 3: Input Gate



This gate decides **what new information should be added** to the cell state.

- **sigmoid ( $\sigma$ )** function is used to decide **how much of the new information** is important.
- **tanh** function is used to generate **new candidate values** to be added.

$$i_t = \sigma(W_i \cdot [h_{t-1}, X_t] + b_i)$$

$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, X_t] + b_C)$$

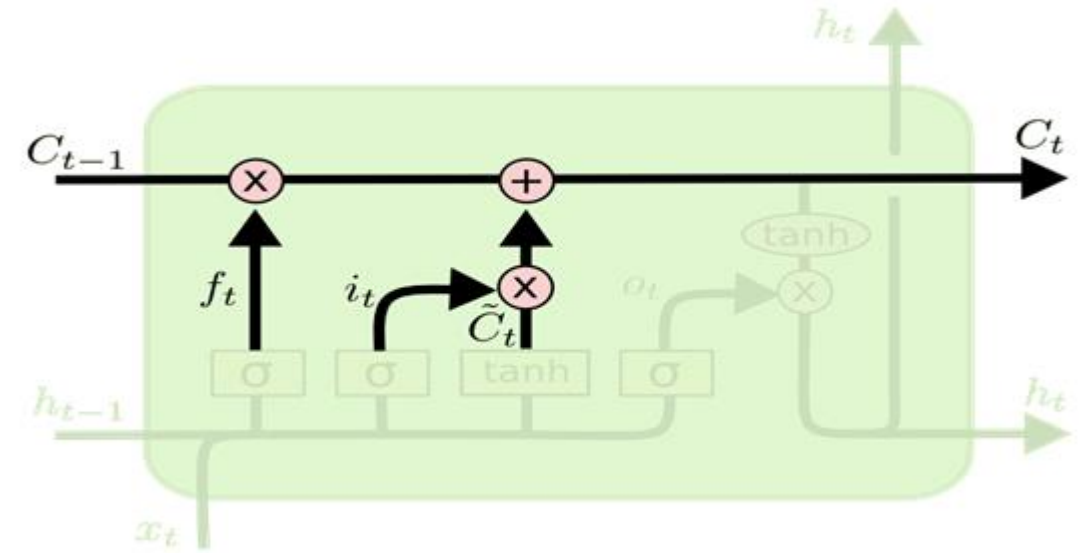
"I love this movie, but it was too long." the input gate might add the phrase **"too long"** to memory since it **influences** the sentiment.

**sigmoid ( $\sigma$ )** activation decides the importance.  
**tanh** regulates new values.

## Step 4: Update Cell State

- The forget gate removes old memory.
- The input gate adds new memory.
- The updated cell state is:

$$C_t = f_t \cdot C_{t-1} + i_t \cdot \tilde{C}_t$$



The LSTM carefully updates its memory instead of overwriting everything like a regular RNN.

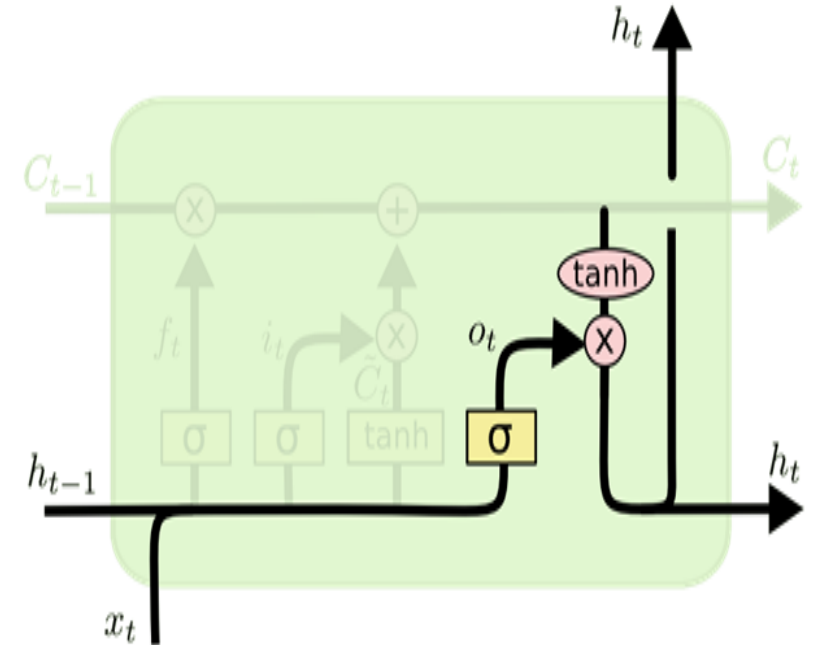


## Step 5: Output Gate

- This gate decides **what information should be sent as the output** for this **time step**.
- Uses **sigmoid ( $\sigma$ )** function to control the output.
- Uses **tanh** function to scale the cell state to a reasonable range.

$$o_t = \sigma(W_o \cdot [h_{t-1}, X_t] + b_o)$$

$$h_t = o_t \cdot \tanh(C_t)$$



## Step 6: Passing Information to the Next Time Step

The hidden state ( $h_t$ ) is sent forward to the next LSTM cell.

The cell state ( $C_t$ ) is passed along with it.

This process repeats for all words in the sentence.

**LSTM prevents vanishing gradient problem by controlling memory updates.**

# Mathematical Representation of an LSTM Unit

Each LSTM unit processes an input  $x_t$  (at time  $t$ ) along with the previous hidden state  $h_{t-1}$  and cell state  $C_{t-1}$ .

## Forget Gate:

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$$

- Uses a **sigmoid function** ( $\sigma$ ) to decide what information to forget.

$f_t \rightarrow$  Forget gate activation (values between 0 and 1).

$\sigma \rightarrow$  Sigmoid activation function (squashes values between 0 and 1).

$W_f \rightarrow$  **Forget gate weight matrix** (learned during training).

$h_{t-1} \rightarrow$  Previous hidden state (short-term memory).

$x_t \rightarrow$  Current input (new data at time step  $t$ ).

$b_f \rightarrow$  **Forget gate bias** (learned during training).

## Example

Previous hidden state:  $h_{t-1} = \begin{bmatrix} 0.5 \\ 0.8 \end{bmatrix}$

Current input:  $x_t = \begin{bmatrix} 0.2 \\ 0.3 \end{bmatrix}$

Forget gate weight matrix:  $W_f = \begin{bmatrix} 0.1 & 0.4 \\ 0.2 & 0.3 \end{bmatrix}$

Forget gate bias:  $b_f = \begin{bmatrix} 0.05 \\ 0.1 \end{bmatrix}$

concatenate  $h_{t-1}$  and  $x_t$ :  $[h_{t-1}, x_t] = \begin{bmatrix} 0.5 \\ 0.8 \\ 0.2 \\ 0.3 \end{bmatrix}$

$$W_f \cdot [h_{t-1}, x_t] = \begin{bmatrix} 0.51 \\ 0.47 \end{bmatrix}$$

add the bias:  $\begin{bmatrix} 0.51 \\ 0.47 \end{bmatrix} + \begin{bmatrix} 0.05 \\ 0.1 \end{bmatrix} = \begin{bmatrix} 0.56 \\ 0.57 \end{bmatrix}$

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

**Apply Sigmoid Activation**

$$\sigma(0.56) = \frac{1}{1 + e^{-0.56}} \approx 0.636$$

$$\sigma(0.57) = \frac{1}{1 + e^{-0.57}} \approx 0.638$$

forget gate output:  $f_t = \begin{bmatrix} 0.636 \\ 0.638 \end{bmatrix}$

0.636 → Retains 63.6% of past information in the first memory unit.

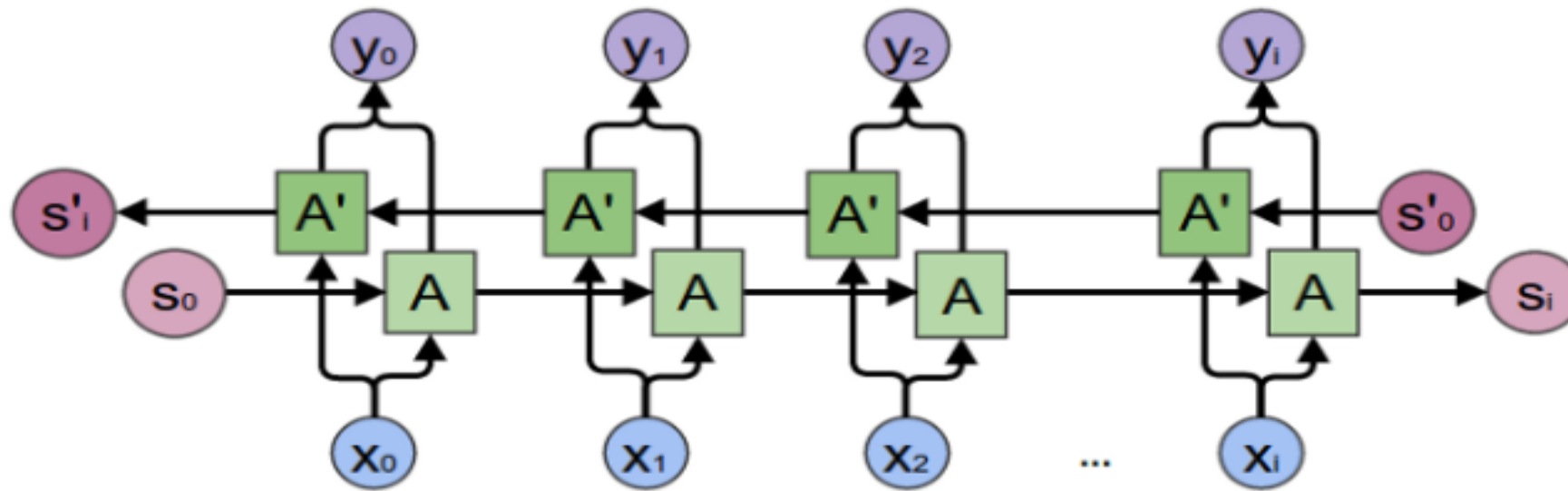
0.638 → Retains 63.8% of past information in the second memory unit.

# Bidirectional LSTM (BiLSTM)

It is an extension of the traditional LSTM that involves **two LSTMs running in parallel**

- one processing the input sequence from start to end (**forward direction**)
- the other processing the sequence from end to start (**backward direction**).
- **Combined Output:** The outputs of both the forward and backward LSTMs are **concatenated** to form the **final output**.

# Bidirectional LSTM



Input Sequence ( $X_0, X_1, \dots, X_i$ ):

Forward LSTM (A)

Backward LSTM ( $A'$ )

Hidden States ( $s_0, s_1, \dots, s_i$ ) and ( $s'_0, s'_1, \dots, s'_i$ ):

# Bi LSTM

## Advantages

Captures long-term dependencies

Improved performance

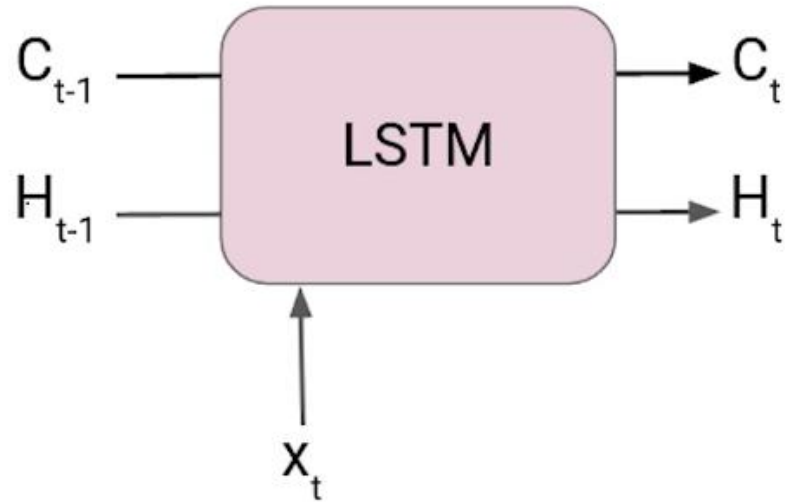
## Disadvantages

High computational cost

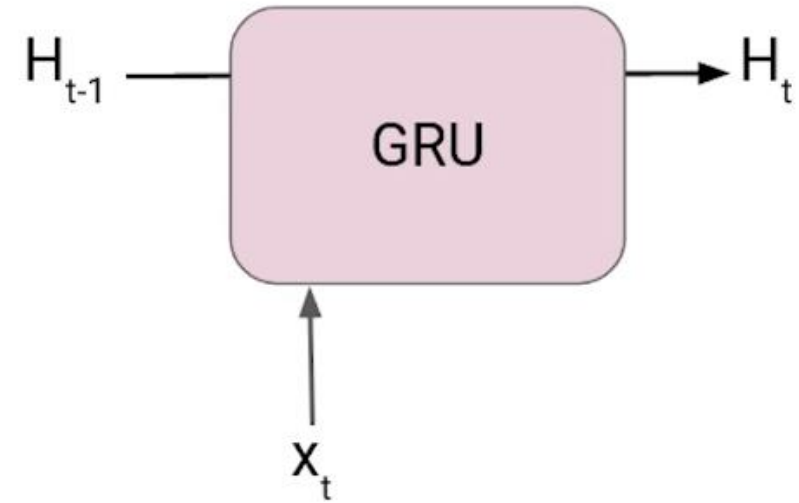
Requires large amounts of data

Difficult to interpret

# Gated Recurrent Unit(GRU)



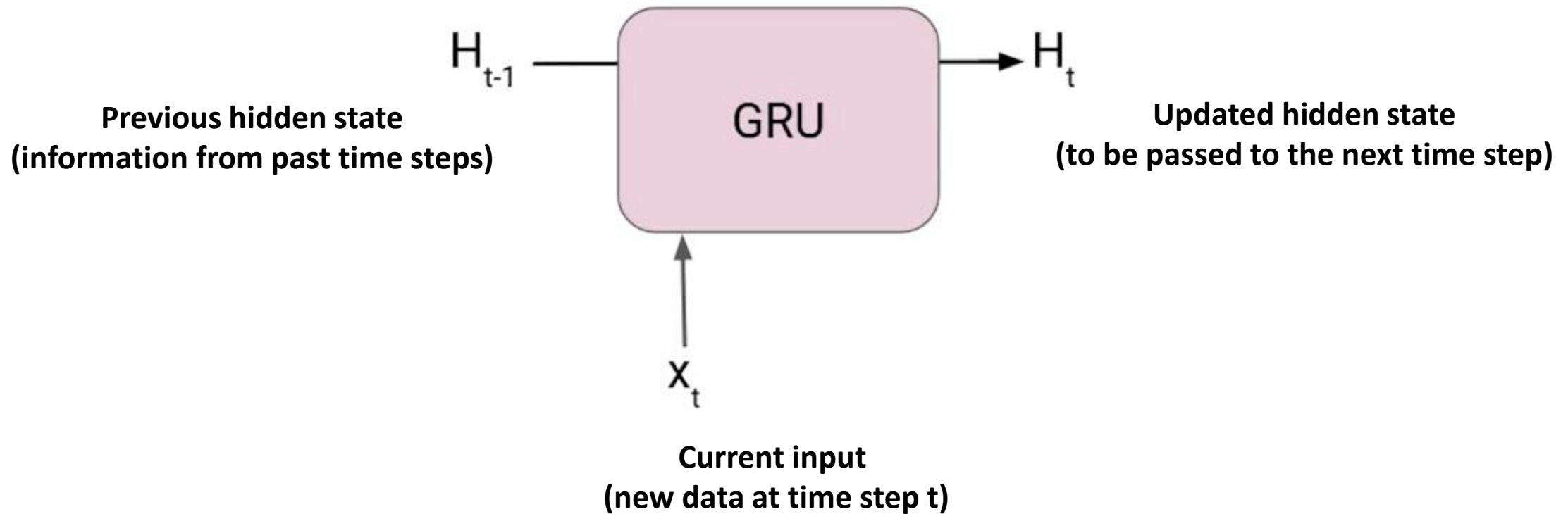
- 3 Gates
- Input, Output, and Forget
- Separate cell state  $C_t$  and Hidden state  $H_t$



- 2 Gates
- Update and Reset
- Single Hidden state  $H_t$



# Gated Recurrent Unit(GRU)



# What Happens Inside the GRU Block?

## Reset Gate ( $r_t$ ):

- Controls how much of the past information ( $H_{t-1}$ ) should be **forgotten**.
- If  $r_t$  is 0, past information is ignored.
- If  $r_t$  is 1, past information is retained.

$$r_t = \sigma(W_r H_{t-1} + U_r X_t + b_r)$$

## Example Values:

- Previous hidden state  $H_{t-1} = 0.6$
- Current input (word representation)  $X_t = 1.0$
- Weights:  $W_r = 0.5$ ,  $U_r = 0.2$
- Bias:  $b_r = 0.1$

$$r_t = \sigma(0.5 \times 0.6 + 0.2 \times 1.0 + 0.1)$$

$$r_t = \sigma(0.3 + 0.2 + 0.1) = \sigma(0.6) = 0.645$$

**This means 64.5% of the past is used.**

### Update Gate ( $z_t$ ):

- Controls how much of the new information ( $X_t$ ) should be **added**.
- If  $z_t$  is **0**, the hidden state is mostly updated with **new information**.
- If  $z_t$  is **1**, the hidden state **retains more of the past**.

$$z_t = \sigma(W_z H_{t-1} + U_z X_t + b_z)$$

### Example Values:

- Weights:  $W_z = 0.3, U_z = 0.4$
- Bias:  $b_z = 0.05$

$$z_t = \sigma(0.3 \times 0.6 + 0.4 \times 1.0 + 0.05)$$

$$z_t = \sigma(0.18 + 0.4 + 0.05) = \sigma(0.63) = 0.653$$

**This means 65.3% of the old hidden state is kept.**

## New Hidden State Calculation:

- A candidate hidden state ( $\tilde{H}_t$ ) is computed based on  $X_t$  and the reset gate.
- The **final hidden state**  $H_t$  is a weighted combination of the old hidden state ( $H_{t-1}$ ) and the candidate hidden state, controlled by the **update gate**.

$$\tilde{H}_t = \tanh(W_h(r_t \cdot H_{t-1}) + U_h X_t + b_h)$$

1. Calculate  $r_t \cdot H_{t-1}$ :

$$0.645 \times 0.6 = 0.387$$

2. Compute  $W_h \tilde{H}_t$ :

- $W_h = 0.7, U_h = 0.5, b_h = 0.2$

$$\tilde{H}_t = \tanh(0.7 \times 0.387 + 0.5 \times 1.0 + 0.2)$$

$$\tilde{H}_t = \tanh(0.27 + 0.5 + 0.2) = \tanh(0.97) = 0.75$$

Compute Final Hidden State  $H_t$ :

---

$$H_t = (1 - z_t)\tilde{H}_t + z_t H_{t-1}$$

$$H_t = (1 - 0.653) \times 0.75 + 0.653 \times 0.6$$

Compute Final Hidden State  $H_t$ :

$$H_t = (0.347 \times 0.75) + (0.653 \times 0.6)$$

$$H_t = 0.260 + 0.392 = 0.652$$

**Final Hidden State:  $H_t=0.652$**

# LSTM Vs GRU

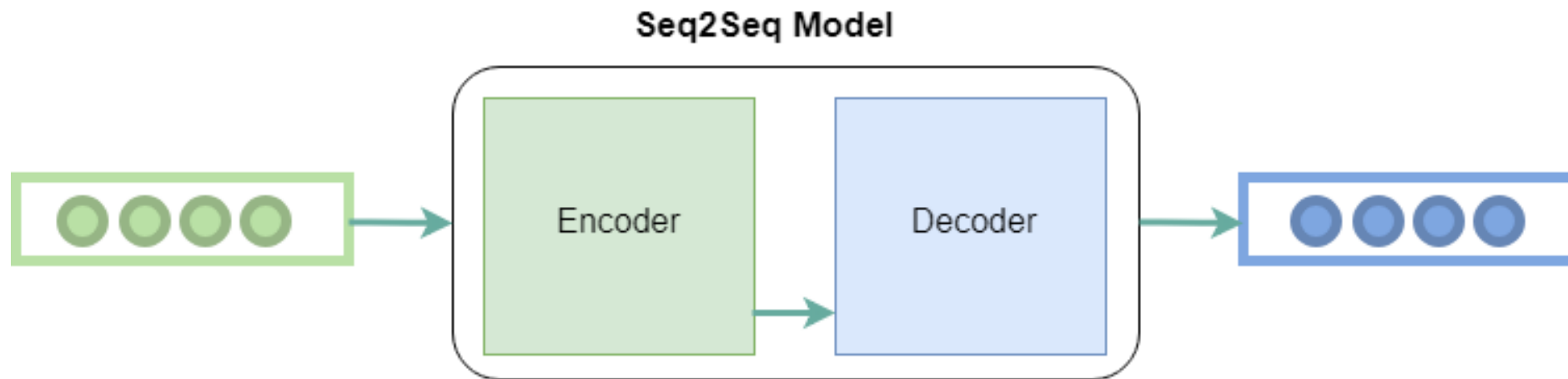
| Feature        | LSTM (Notebook)                                                                | GRU (Sticky Note)                                  |
|----------------|--------------------------------------------------------------------------------|----------------------------------------------------|
| Memory Storage | Long-term memory (Cell State $C_t$ ) + Short-term memory (Hidden State $H_t$ ) | Only one memory state (Hidden State $H_t$ )        |
| Control        | Uses <b>three gates</b> (Forget, Input, Output)                                | Uses <b>two gates</b> (Reset, Update)              |
| Complexity     | More powerful but slower                                                       | Simpler and faster                                 |
| Use Case       | Best for complex, long-term dependencies (e.g., essays)                        | Best for short-term dependencies (e.g., summaries) |

# When to use ?

| Model              | Best for Sentence Length                      | Example Sentence                                                                                                                |
|--------------------|-----------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------|
| RNN<br>(SimpleRNN) | Short sentences (1-10 words)                  | "This movie was great!"                                                                                                         |
| GRU                | Medium sentences (10-30 words)                | "The movie started slow, but the action scenes were thrilling!"                                                                 |
| LSTM               | Long sentences (30+ words) or full paragraphs | "Despite its slow start, the film gradually built suspense and delivered a thrilling climax that left the audience mesmerized." |

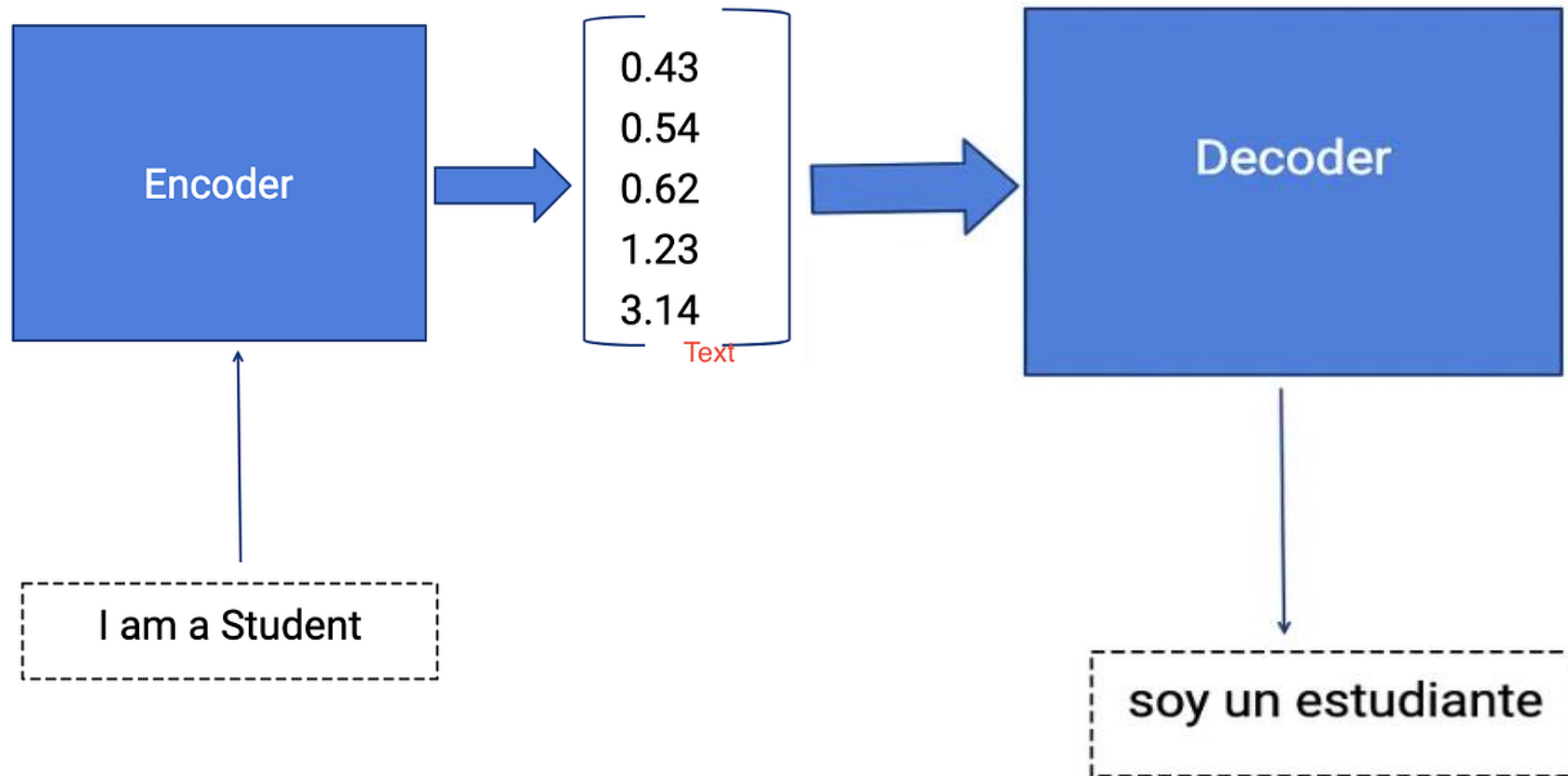
# Sequence to Sequence(Seq2Seq) Model

- Sequence-to-Sequence model, is a machine learning architecture designed for tasks involving **sequential data**.
- The architecture consists of **two** fundamental components: an **encoder** and a **decoder**.
- The encoder processes the input sequence and transforms it into a fixed-size hidden representation. The decoder uses the hidden representation to generate output sequence.

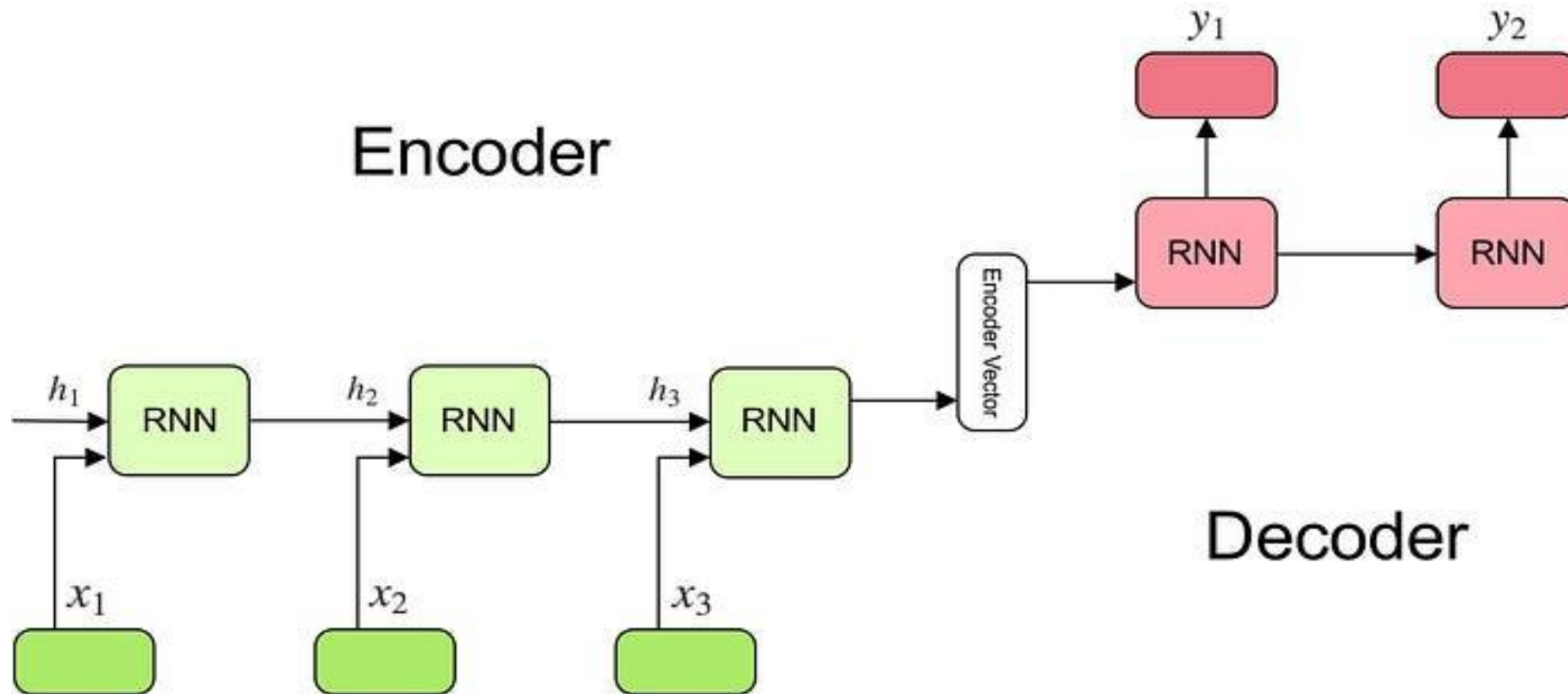




# Machine Translation



# Seq2Seq using RNN



# Example: Machine Translation

- Translate "How are you?" into French.

| Time Step | Encoder (English)                | Decoder (French) |
|-----------|----------------------------------|------------------|
| $t = 1$   | "How" $\rightarrow h_1$          | -                |
| $t = 2$   | "are" $\rightarrow h_2$          | -                |
| $t = 3$   | "you" $\rightarrow h_3$          | -                |
| Final     | Encoder Vector $\rightarrow h_3$ | -                |
| $t = 1$   | -                                | "Comment"        |
| $t = 2$   | -                                | "ça"             |
| $t = 3$   | -                                | "va"             |

"How are you?"  $\rightarrow$  "Comment ça va?"

# Which Seq2Seq Model Should we Use?

- **Use RNN** → If sentences are short and simple model.
- **Use LSTM** → for best accuracy for long sentences.
- **Use GRU** → for **LSTM-like performance** but with faster training.

# What is Teacher Forcing?

- **Teacher Forcing** is a **training technique** used in **sequence-to-sequence (Seq2Seq) models** (like RNN, LSTM, and GRU) where the actual target word from the dataset is fed as the next input to the decoder **instead of using the predicted word** from the previous step.
- **Without teacher forcing**, the model would rely only on its own predicted words during training. If a mistake is made early, that error **propagates through the sequence**, making later predictions worse.
- **Teacher Forcing speeds up training** by ensuring that the model learns correct mappings efficiently.
- However, it can cause problems during testing because the model never learns how to correct its mistakes when it makes wrong predictions.

# Applications

- Google's Machine Translation
- Question answering chatbots
- Speech recognition
- Time Series Application etc.,

**Seq2Seq models have been widely used in NLP\_tasks due to their ability to handle variable-length input and output sequences.**